UTEPSA · SAL-301 ALGORITMOS Y PROGRAMACIÓN

Diseño de la solución

Clases, diagramas UML, pseudocódigo y estructura del proyecto C#

Proyecto: As bajo la manga — Sistema web de reservas para barbería
Docente: Ing. Gloria M. Gonzales Fernandez
Estudiante: Eduard Gonzales
Octubre de 2026

Presentación 2 — Diseño de la solución (12/10)

El cronograma pide para esta presentación **diagrama UML + pseudocódigo + estructura del proyecto C#** (POO, clases, atributos, métodos, constructores y UML). Este documento parte del análisis de la Etapa 2 (requerimientos, reglas, entradas-procesos-salidas, algoritmos A1 a A5.1 y sus diagramas de flujo) y lo convierte en un diseño orientado a objetos que **ya está programado y compila**: los nombres de clases, atributos y métodos del UML son los mismos del código.

> Documentos relacionados: `analisis/Analisis_Diseno_Barberia.docx` (Etapa 2) y el código en `codigo/` (solución `BarberiaAsBajoLaManga.slnx`).

1. Resumen del análisis

Tema	Decisión de la Etapa 2
Problema	Las citas se coordinan por WhatsApp, llamadas o un cuaderno: hay choques de horario, citas perdidas y no hay confirmación.
Solución	Sistema web en C# (ASP.NET Core MVC): el cliente reserva en 5 pasos (servicio, peluquero, fecha y hora, datos, confirmar); la recepcionista y el administrador gestionan la agenda desde un panel con inicio de sesión.
Datos	Supabase (PostgreSQL). Correo de confirmación con Brevo. Registro de errores y exportaciones en archivos .txt.
Regla central	Dos citas no se superponen: [horaInicio, horaFin) choca si `horaInicio < horaFinExistente` y `horaFin > horaInicioExistente` (RN-06).
Algoritmo central	A1 Calcular horarios disponibles, con el subalgoritmo A1.1 HayChoque. Complejidad O(b × r).
Requerimientos	23 funcionales (16 de prioridad Alta para la V1) y 12 no funcionales.

2. Arquitectura y estructura de la solución

La solución mantiene la arquitectura de **2 proyectos** usada en clase, más un proyecto de pruebas:

Proyecto	Tipo	Contenido
`logica` (Logica.csproj)	Biblioteca de clases	Clases del dominio, enumeraciones, gestores con los algoritmos, validaciones, seguridad, acceso a datos, correo, registro de errores y la excepción propia. No depende de la web.
`presentaciones/WebBarberia`	ASP.NET Core MVC	Controladores, vistas Razor y CSS. Usa la capa `logica`.
`pruebas/Logica.Pruebas`	xUnit	23 pruebas automáticas de los algoritmos y reglas (todas pasan).

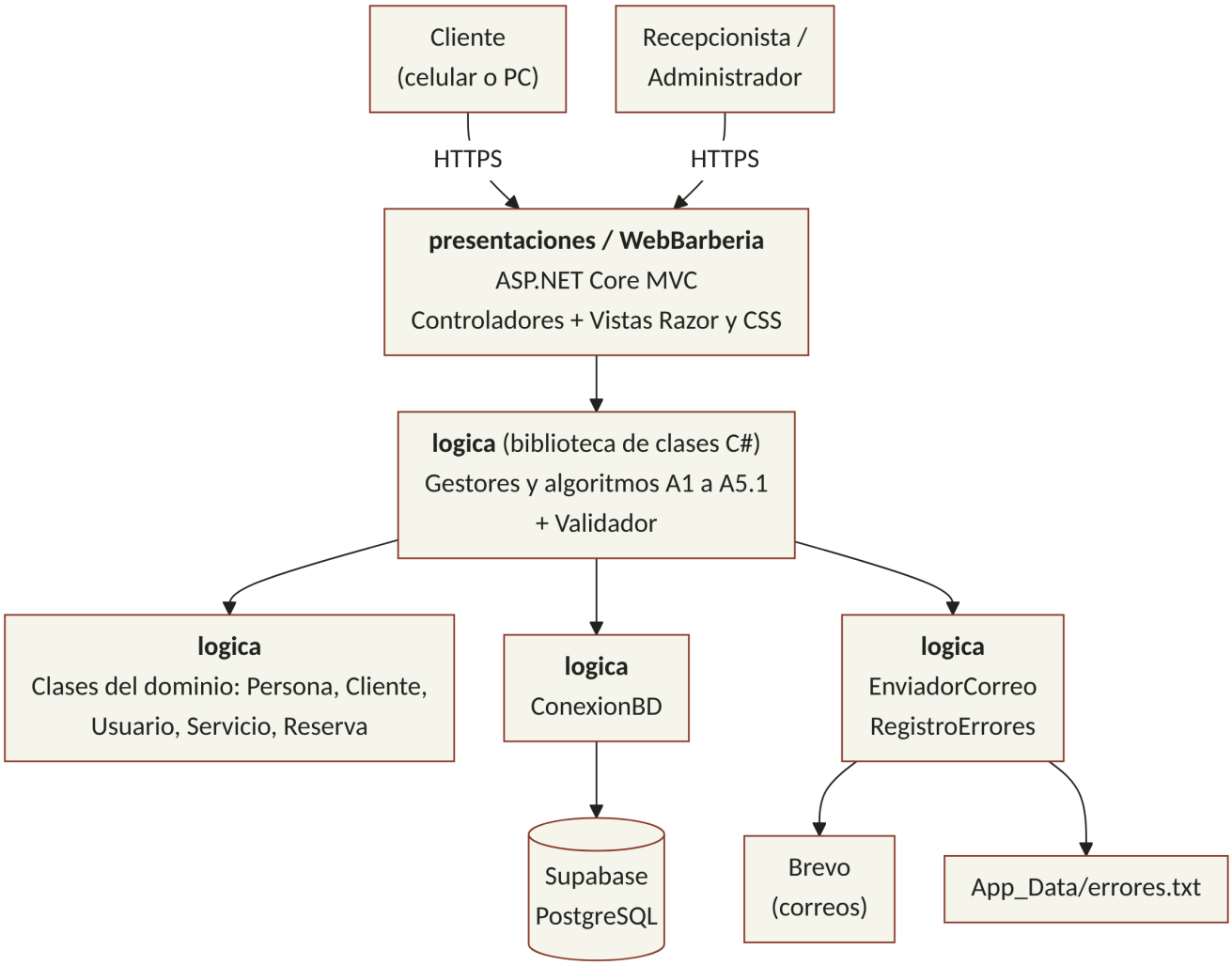


Figura 1. Arquitectura de la solución

3. Identificación de clases, atributos y métodos

Criterio de diseño (POO): cada clase del dominio tiene **atributos privados**, un **constructor** que valida los datos y **propiedades de solo lectura** para consultarlos; los cambios de estado se hacen solo con **métodos públicos** que aplican las reglas del negocio (encapsulamiento). `Persona` es una clase **abstracta** que heredan `Cliente` y `Usuario`.

3.1 Clases del dominio

Clase	Responsabilidad
Persona (abstracta)	Datos comunes de una persona: nombre y apellido validados.
Cliente	Cliente de la barbería, identificado por su teléfono (RN-11).
Usuario	Personal con inicio de sesión (Administrador o Recepcionista), con bloqueo tras 3 intentos fallidos (RN-18).
Peluquero	Peluquero que atiende las citas; cada uno tiene su propia agenda (RN-21). Se desactiva, no se elimina.
Servicio	Servicio que se ofrece, con duración múltiplo de 30 y precio (RN-04). Se desactiva, no se elimina (RN-16).
Reserva	Cita de un cliente para un servicio con un peluquero; controla sus cambios de estado (RN-13, RN-14) y la superposición (RN-06).

Clase	Responsabilidad
Turno y HorarioAtencion	Horario de atención por turnos y parámetros de la agenda (RN-02, RN-03, RN-08, RN-09).
DatosReserva	Datos que llegan del formulario de reserva.
Reporte y ConteoServicio	Resultado del reporte por rango de fechas (A5) y pares que se ordenan por burbuja (A5.1).
EstadoReserva, Rol, OrigenReserva	Enumeraciones: valores fijos del sistema.

Persona (abstracta)

Miembro	Descripción
`-nombre: string`, `-apellido: string`	Nombre y apellido validados (2 a 50 letras).
`#Persona(nombre, apellido)`	Constructor protegido: solo lo usan las clases hijas.
`+CambiarNombre(nombre, apellido): void`	Valida y cambia el nombre.

Cliente : Persona

Miembro	Descripción
`-id: int`	Identificador en la base de datos.
`-telefono: string`	8 dígitos sin +591; único (RN-11).
`-correo: string?`	Opcional (vacío si no tiene).
`-fechaRegistro: DateTime`	Hora de Bolivia en que se registró.
`+Cliente(id, nombre, apellido, telefono, correo, fechaRegistro)`	Constructor: valida teléfono y correo.
`+CambiarTelefono(nuevoTelefono): void` / `+CambiarCorreo(nuevoCorreo): void`	Validan y cambian el dato.
`+CompletarCorreo(nuevoCorreo): void`	En la reserva web solo completa el correo si estaba vacío (no sobrescribe datos).
`+AsignarId(nuevoId): void`	Guarda el id que devuelve la base de datos.

Usuario : Persona

Miembro	Descripción
`+MaximoIntentos = 3`, `+MinutosBloqueo = 5`	Constantes de RN-18.
`-id`, `-nombreUsuario`, `-rol: Rol`, `-activo: bool`	Datos de la cuenta.
`-contrasenaHash: string`	"sal:hash" calculado con PBKDF2; nunca la contraseña (RNF-02).
`-intentosFallidos: int`, `-bloqueadoHasta: DateTime?`	Control del bloqueo; se guardan en la base de datos.
`+EstaBloqueado(ahora): bool`	Verdadero si bloqueadoHasta es posterior a ahora.
`+RegistrarIntentoFallido(ahora): bool`	Suma un intento; al tercero bloquea 5 minutos.
`+ReiniciarIntentos(): void`	Al entrar bien: pone el contador en 0 y quita el bloqueo.
`+CambiarContrasena(nueva): void`	Exige contraseña segura y guarda su hash.
`+CambiarRol(nuevoRol)`, `+Activar()`, `+Desactivar()`	Administración de la cuenta.

Peluquero : Persona

Miembro	Descripción
`-id: int`	Identificador en la base de datos.
`-especialidad: string`	Opcional, máximo 60 caracteres.
`-activo: bool`	Si es falso, no aparece en la web (borrado lógico).
`+Peluquero(id, nombre, apellido, especialidad, activo)`	Constructor: valida nombre, apellido y especialidad.
`+CambiarEspecialidad(nuevaEspecialidad): void`	Valida y cambia la especialidad.
`+Activar()`, `+Desactivar()`, `+AsignarId(nuevoId)`	Borrado lógico e id de la base de datos.

Servicio

Miembro	Descripción
`-id`, `-nombre`, `-descripcion`, `-imagen`	Datos del catálogo.
`-duracionMin: int`, `-precio: decimal`	Validados: múltiplo de 30 entre 30 y 180; precio mayor que 0.
`-activo: bool`	Si es falso, no aparece en la web.
`+Servicio(...)`, `+Modificar(...)`	Constructor y modificación con validación.
`+Activar()`, `+Desactivar()`	Borrado lógico.

Reserva

Miembro	Descripción
`-cliente: Cliente`, `-servicio: Servicio`, `-peluquero: Peluquero`, `-creadoPor: Usuario?`	Asociaciones (ver UML).
`-fecha: DateOnly`, `-horaInicio`, `-horaFin: TimeOnly`	Cuándo es la cita.
`-precio: decimal`	Copiado del servicio al crearla (RN-15).
`-estado: EstadoReserva`, `-origen: OrigenReserva`, `-notas`	Estado, web o manual, y notas.
`-correoEnviado: bool?`, `-creadoEn: DateTime`	Resultado del correo (vacío = sin correo) y fecha de registro.
`+Reserva(cliente, servicio, peluquero, fecha, horaInicio, origen, notas, hayCorreo, creadoEn, creadoPor)`	Reserva nueva: calcula horaFin, copia el precio y queda Confirmada.
`+Inicio`, `+OcupaHorario`	Propiedades calculadas: fecha y hora de inicio; si ocupa horario (Confirmada o Atendida).
`+SeSuperponeCon(inicio, fin): bool`	Aplica la fórmula de RN-06.
`+Cancelar(ahora)`, `+MarcarAtendida(ahora)`, `+MarcarNoAsistio(ahora)`, `+Reprogramar(...)`	Cambios de estado con RN-13 y RN-14; si no se cumplen, lanzan una excepción.
`+RegistrarEnvioCorreo(enviado)`, `+AsignarId(nuevoId)`	Guarda el resultado del correo y el id que devuelve la base de datos.

3.2 Clases de la lógica

Clase	Métodos principales	Algoritmo / requisito
GestorReservas	`CalcularHorariosDisponibles`, `HayChoque`, `RegistrarReserva`, `CambiarEstado`, `Reprogramar`, `ObtenerAgenda`, `ObtenerPorCliente`, `AgendaATexto`	A1, A1.1, A2, A4; RF-10, RF-21, RF-24
GestorClientes	`BuscarEnLista` (búsqueda lineal), `Listar`, `Registrar`, `Modificar`, `Eliminar`, `Historial`	RF-16, RF-17
GestorPeluqueros	`ListarActivos`, `ListarTodos`, `ObtenerPorId`, `Registrar`, `Modificar`, `CambiarEstado`	RF-24, RF-25, RN-21
GestorServicios	`Referenciales`, `ListarActivos`, `ListarTodos`, `Registrar`, `Modificar`, `CambiarEstado`	RF-01, RF-18
GestorUsuarios	`IniciarSesion`, `VerificarIntento`, `PuedeDesactivar`, `Crear`, `CrearAdministradorInicialSiNoHay`	A3; RF-08, RF-09, RF-19
GeneradorReportes	`Generar`, `OrdenarBurbuja`, `ExportarTxt`	A5, A5.1; RF-20
Validador (estática)	`EsNombreValido`, `NormalizarTelefono`, `EsTelefonoValido`, `EsCorreoValido`, `EsContrasenaSegura`, `ValidarDatosReserva`	Validaciones (sección 6 del análisis)
Seguridad (estática)	`CalcularHash`, `VerificarHash`	RNF-02
Reloj (estática)	`AhoraBolivia`, `HoyBolivia`	RN-10
RegistroErrores	`Registrar`, `LeerUltimas`	RF-22, RF-23 (archivo de texto)
ConexionBD	`Abrir`	Conexión a Supabase
EnviadorCorreo	`EnviarConfirmacion`	RF-07, RN-20
HorarioNoDisponibleException	Constructor	Excepción propia (RN-06, RNF-07)

Los métodos que necesitan la base de datos (por ejemplo `Listar` o `ObtenerReservasDelDia`) ya tienen su firma definida y se completan en la implementación (13–16/10). Los algoritmos tienen además una versión **sin base de datos** (estática), que es la que se prueba.

4. Diagramas UML de clases

Notación: `-` privado, `+` público, `#` protegido; subrayado = estático; «abstract», «enumeration» y «static» son estereotipos. Flecha con triángulo hueco = herencia; flecha simple = asociación navegable (con su nombre de rol y multiplicidad en cada extremo); rombo relleno = composición; línea punteada = dependencia («usa» o «lanza»).

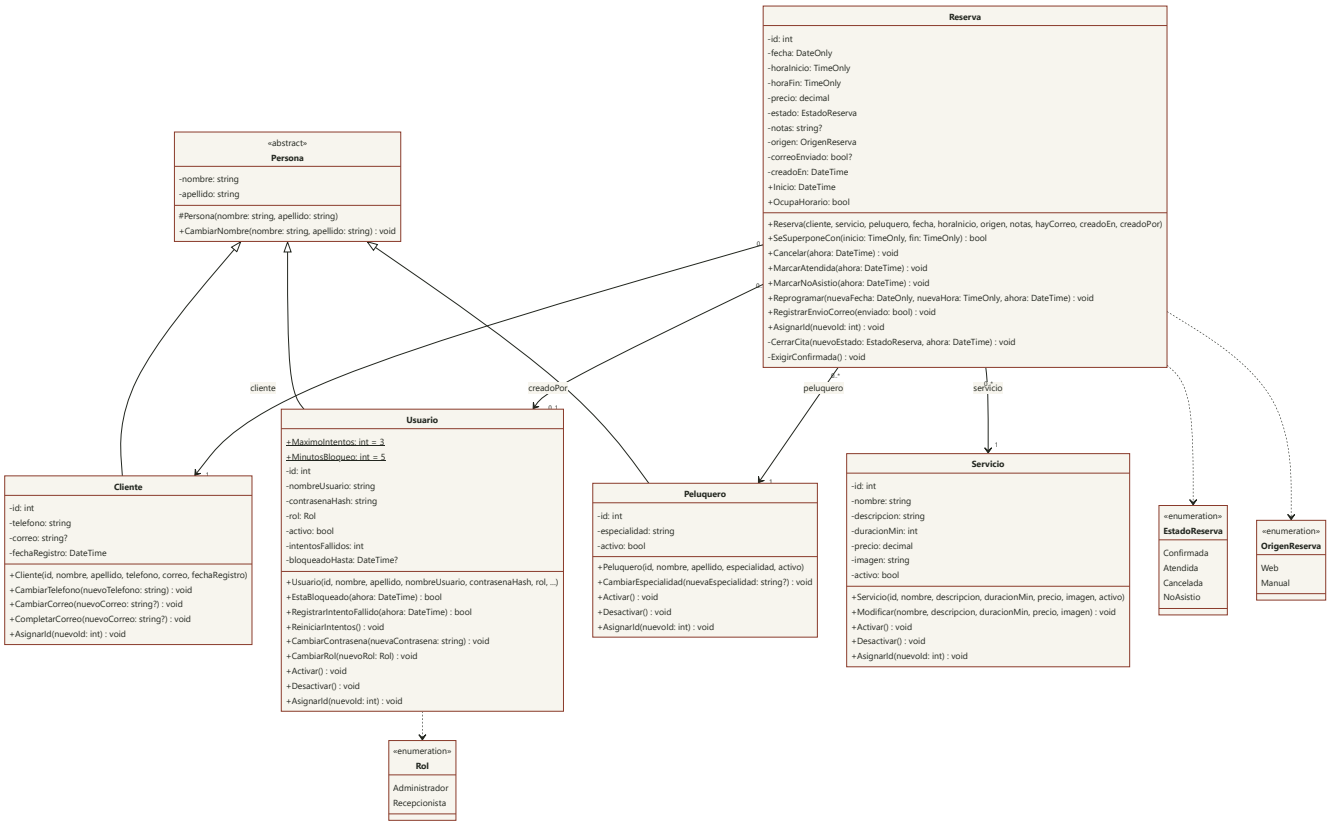


Figura 2. Modelo del dominio: herencia, asociaciones y multiplicidades

Relación	Tipo	Multiplicidad	Significado
Cliente, Usuario, Peluquero → Persona	Herencia	—	Cliente, Usuario y Peluquero **son** personas: heredan nombre, apellido y CambiarNombre.
Reserva → Peluquero (peluquero)	Asociación	0..* a 1	Cada reserva la atiende un peluquero; un peluquero atiende muchas reservas (RN-21).
Reserva → Cliente (cliente)	Asociación	0..* a 1	Cada reserva es de un cliente; un cliente puede tener muchas reservas.
Reserva → Servicio (servicio)	Asociación	0..* a 1	Cada reserva es de un servicio; un servicio aparece en muchas reservas.
Reserva → Usuario (creadoPor)	Asociación	0..* a 0..1	Las reservas manuales guardan qué usuario las registró; las web no tienen usuario.
Reserva ⇢ EstadoReserva, OrigenReserva; Usuario ⇢ Rol	Dependencia	—	Usan esas enumeraciones como tipo de sus atributos.
HorarioAtencion ◆ Turno	Composición	1 a 2	El horario se compone de 2 turnos (mañana y tarde).
Reporte ◆ ConteoServicio	Composición	1 a 0..*	El reporte contiene la lista de servicios ordenada.

Relación	Tipo	Multiplicidad	Significado
Gestores → ConexionBD, EnviadorCorreo, RegistroErrores	Asociación	1 a 1	Cada gestor recibe en su constructor los objetos que necesita.
Exception ◁ HorarioNoDisponibleException	Herencia	—	Excepción propia del sistema.

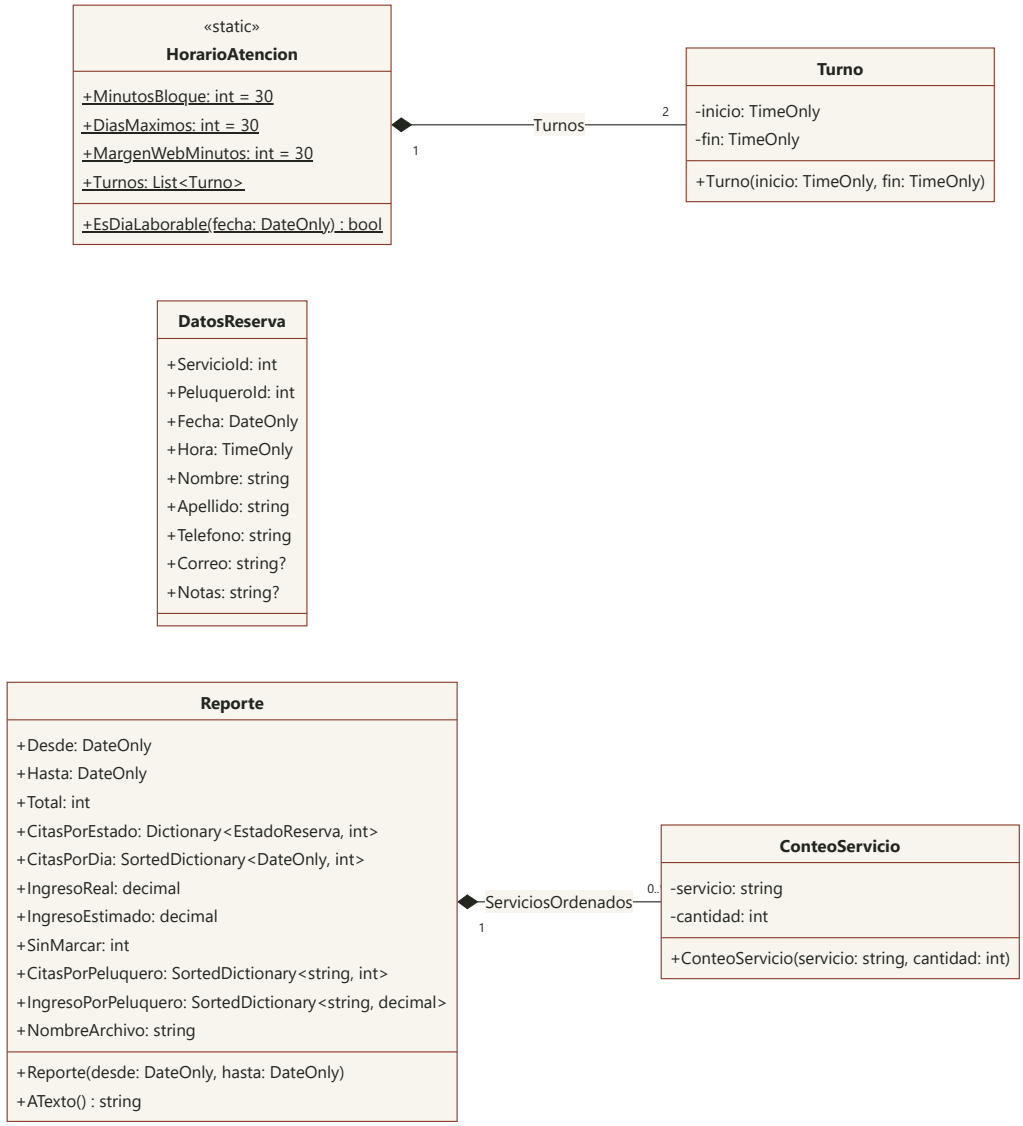


Figura 3. Horario de atención, datos de la reserva y reporte

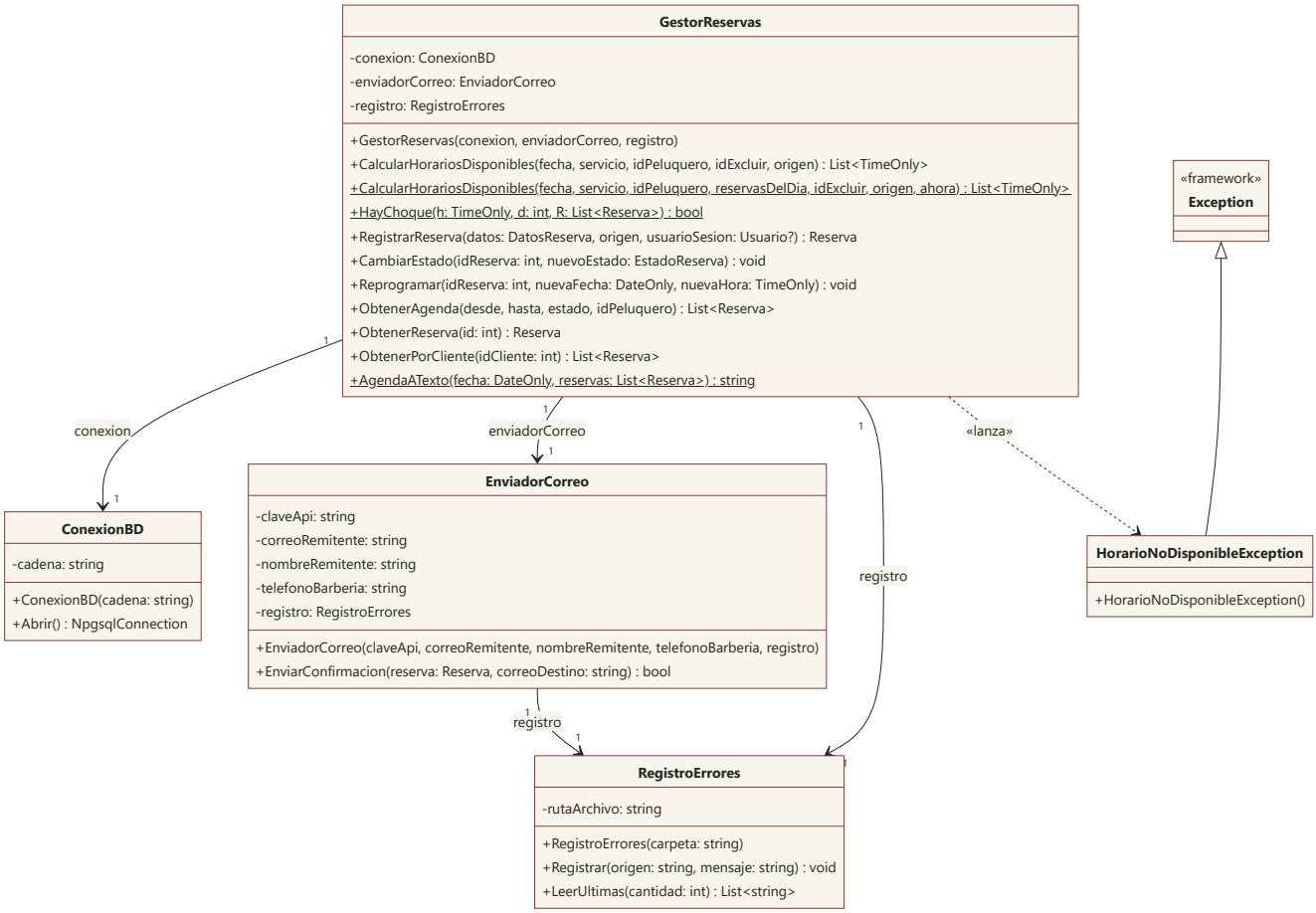
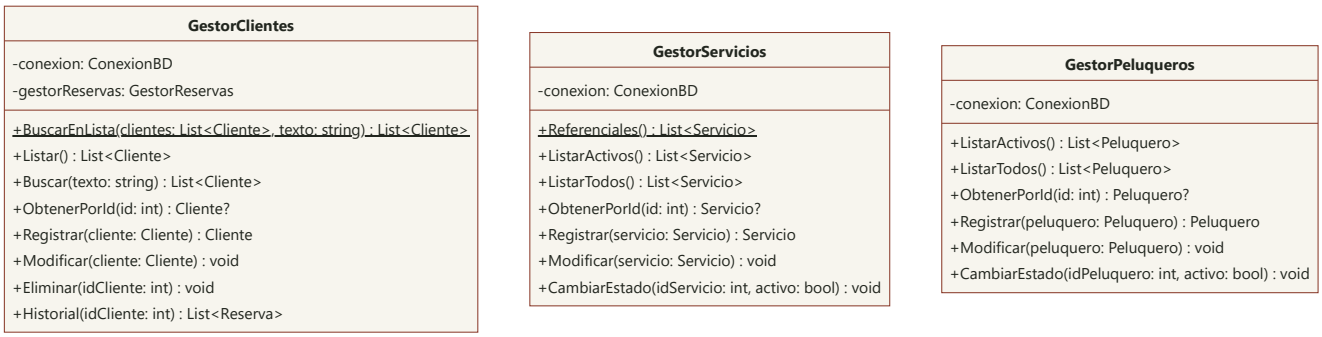


Figura 4. GestorReservas y sus colaboradores


GestorClientes
-conexion: ConexionBD
-gestorReservas: GestorReservas
+BuscarEnLista(clientes: List<Cliente>, texto: string) : List<Cliente>
+Listar() : List<Cliente>
+Buscar(texto: string) : List<Cliente>
+ObtenerPorId(id: int) : Cliente?
+Registrar(cliente: Cliente) : Cliente
+Modificar(cliente: Cliente) : void
+Eliminar(idCliente: int) : void
+Historial(idCliente: int) : List<Reserva>
GestorServicios
-conexion: ConexionBD
+Referenciales() : List<Servicio>
+ListarActivos() : List<Servicio>
+ListarTodos() : List<Servicio>
+ObtenerPorId(id: int) : Servicio?
+Registrar(servicio: Servicio) : Servicio
+Modificar(servicio: Servicio) : void
+CambiarEstado(idServicio: int, activo: bool) : void
GestorPeluqueros
-conexion: ConexionBD
+ListarActivos() : List<Peluquero>
+ListarTodos() : List<Peluquero>
+ObtenerPorId(id: int) : Peluquero?
+Registrar(peluquero: Peluquero) : Peluquero
+Modificar(peluquero: Peluquero) : void
+CambiarEstado(idPeluquero: int, activo: bool) : void

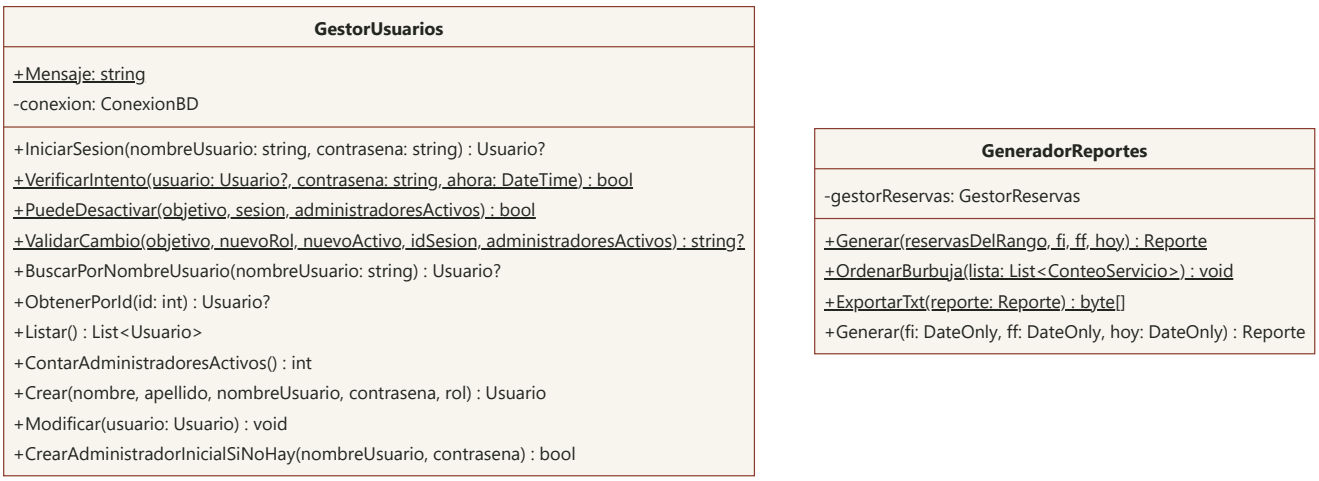


Figura 5. Gestores de clientes, servicios, peluqueros, usuarios y reportes

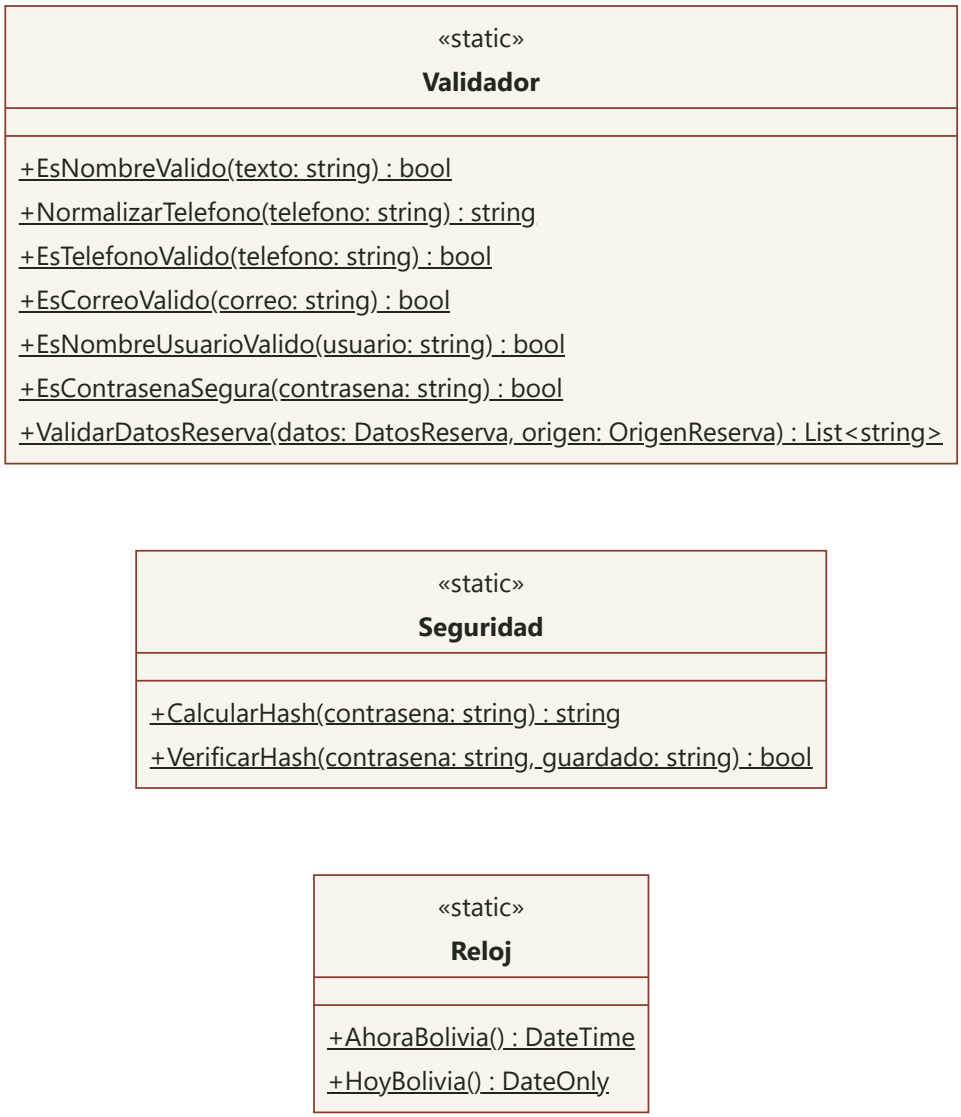
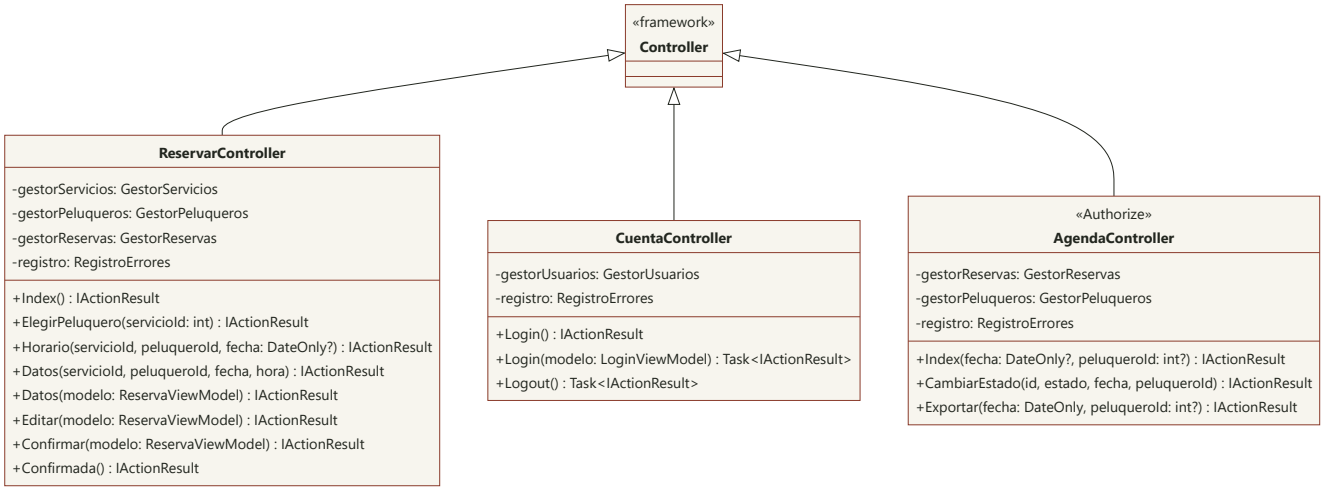
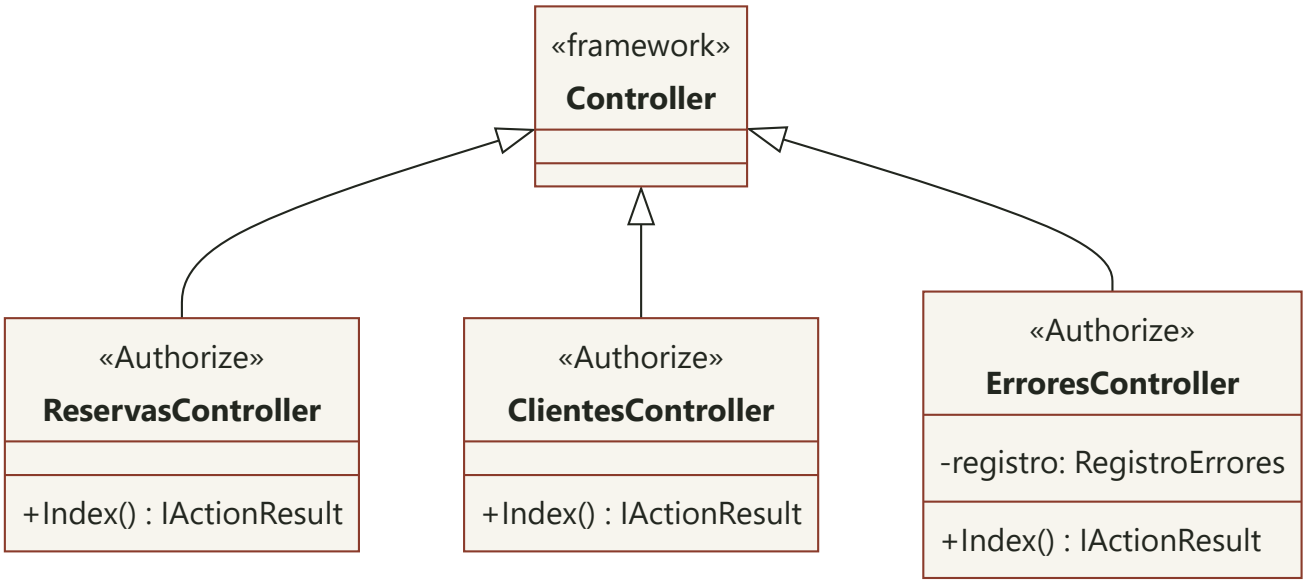


Figura 6. Clases utilitarias estáticas y quién las usa


«framework»
Controller
ReservarController
-gestorServicios: GestorServicios
-gestorPeluqueros: GestorPeluqueros
-gestorReservas: GestorReservas
-registro: RegistroErrores
+Index() : IActionResult
+ElegirPeluquero(servicioId: int) : IActionResult
+Horario(servicioId, peluqueroId, fecha: DateOnly?) : IActionResult
+Datos(servicioId, peluqueroId, fecha, hora) : IActionResult
+Datos(modelo: ReservaViewModel) : IActionResult
+Editar(modelo: ReservaViewModel) : IActionResult
+Confirmar(modelo: ReservaViewModel) : IActionResult
+Confirmada() : IActionResult
CuentaController
-gestorUsuarios: GestorUsuarios
-registro: RegistroErrores
+Login() : IActionResult
+Login(modelo: LoginViewModel) : Task<IActionResult>
+Logout() : Task<IActionResult>
«Authorize»
AgendaController
-gestorReservas: GestorReservas
-gestorPeluqueros: GestorPeluqueros
-registro: RegistroErrores
+Index(fecha: DateOnly?, peluqueroId: int?) : IActionResult
+CambiarEstado(id, estado, fecha, peluqueroId) : IActionResult
+Exportar(fecha: DateOnly, peluqueroId: int?) : IActionResult


«framework»
Controller
«Authorize»
ReservasController
+Index() : IActionResult
«Authorize»
ClientesController
+Index() : IActionResult
«Authorize»
ErroresController
-registro: RegistroErrores
+Index() : IActionResult

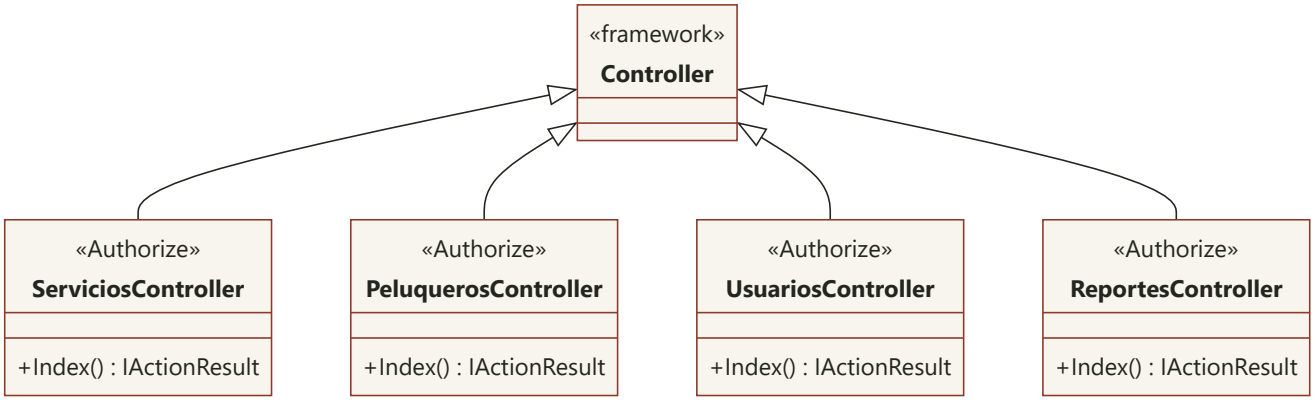


Figura 7. Capa de presentación: controladores MVC

5. Pseudocódigo

> **Notación (estilo PSeInt).** `<-` asignación; `Si … Entonces … SiNo … FinSi`; `Segun … Hacer … FinSegun`; `Mientras … Hacer … FinMientras`; `Para … Hasta … Hacer … FinPara`; `Para Cada x En lista Hacer` (recorre una lista); `Repetir … Hasta Que`; `Funcion r <- Nombre(…)` devuelve r. Para el manejo de errores se usan `Lanzar` (throw) e `Intentar … Capturar` (try/catch), que PSeInt no tiene pero C# sí. Cada pseudocódigo corresponde, línea por línea, al método del mismo nombre en el código.

5.1 A1 — CalcularHorariosDisponibles (GestorReservas)

Es la versión sin base de datos (estática), la que se prueba. La versión de 5 parámetros `CalcularHorariosDisponibles(F, S, P, idExcluir, origen)`, que usan A2 y A4, lee `reservasDelDia` de Supabase y `ahora <- AhoraBolivia()`, y llama a esta.

```
Funcion disponibles <- CalcularHorariosDisponibles(F, S, P, reservasDelDia, idExcluir, origen, ahora)
    Definir disponibles, R Como Lista
    Definir hoy Como Fecha
    Definir limite Como FechaHora
    Definir d Como Entero
    Definir h Como Hora
    Definir fechaValida, choca Como Logico
    disponibles <- ListaVacia()
    hoy <- FechaDe(ahora)
    fechaValida <- F >= hoy Y F <= hoy + 30 dias Y EsDiaLaborable(F)
    Si fechaValida Entonces
        d <- S.DuracionMin
        Si origen = Web Entonces
            limite <- ahora + 30 minutos
        SiNo
            limite <- InicioDelBloqueEnCurso(ahora)   // manual: permite el bloque en curso
        FinSi
        R <- ListaVacia()
        Para Cada r En reservasDelDia Hacer
            Si r.Fecha = F Y r.Peluquero.Id = P Y r.OcupaHorario Y r.Id <> idExcluir Entonces
                Agregar(R, r)
            FinSi
        FinPara
        Para Cada T En HorarioAtencion.Turnos Hacer
            h <- T.Inicio
            Mientras h + d <= T.Fin Hacer
                Si F + h >= limite Entonces                // el bloque todavía no pasó
                    choca <- HayChoque(h, d, R)
                    Si NO choca Entonces
                        Agregar(disponibles, h)
                    FinSi
                FinSi
                h <- h + 30 minutos
            FinMientras
        FinPara
    FinSi
FinFuncion
```

5.2 A1.1 — HayChoque (GestorReservas)

```
Funcion choca <- HayChoque(h, d, R)
    Definir fin Como Hora
    Definir i Como Entero
    fin <- h + d
    choca <- Falso
    i <- 0
    Mientras i < Longitud(R) Y NO choca Hacer
        Si h < R[i].HoraFin Y fin > R[i].HoraInicio Entonces
            choca <- Verdadero
        FinSi
        i <- i + 1
    FinMientras
FinFuncion
```

5.3 A2 — Reserva web (pantalla) y RegistrarReserva (GestorReservas)

```
Algoritmo ReservarCitaWeb                       // la página y su controlador
    Repetir
        Escribir ServiciosActivos()
        Leer S
        Escribir PeluquerosActivos()              // si hay uno solo, se elige solo
        Leer P
        Repetir
            Leer F
            disponibles <- CalcularHorariosDisponibles(F, S, P, 0, Web)
            Si Longitud(disponibles) = 0 Entonces
                Escribir "No hay horarios disponibles para esta fecha"
            FinSi
        Hasta Que Longitud(disponibles) > 0
        Escribir disponibles                      // los demás bloques, tachados
        Leer H
        Repetir
            Leer datos.Nombre, datos.Apellido, datos.Telefono, datos.Correo, datos.Notas
            errores <- ValidarDatosReserva(datos, Web)
            Si Longitud(errores) > 0 Entonces
                Escribir errores
            FinSi
        Hasta Que Longitud(errores) = 0
        Escribir Resumen(S, P, F, H, datos)
        Leer opcion                               // "Confirmar" o "Editar"
    Hasta Que opcion = "Confirmar"
    Intentar
        res <- RegistrarReserva(datos, Web, Nulo)
        Escribir "¡Cita confirmada!", res
    Capturar HorarioNoDisponibleException
        Escribir "Ese horario ya no está disponible, elige otro"
    Capturar otro error
        RegistrarError(error)
        Escribir "No se pudo registrar la reserva, intenta nuevamente"
    FinIntentar
FinAlgoritmo

Funcion res <- RegistrarReserva(datos, origen, usuarioSesion)
    errores <- ValidarDatosReserva(datos, origen)
    Si Longitud(errores) > 0 Entonces
        Lanzar ErrorDeArgumento(errores)            // ArgumentException
    FinSi
    servicio <- ObtenerServicio(datos.ServicioId)
    peluquero <- ObtenerPeluquero(datos.PeluqueroId)
    disponibles <- CalcularHorariosDisponibles(datos.Fecha, servicio, peluquero.Id, 0, origen)
    Si NO Contiene(disponibles, datos.Hora) Entonces
        Lanzar HorarioNoDisponibleException
    FinSi
    Intentar
        res <- GuardarClienteYReserva(datos, servicio, peluquero, origen, usuarioSesion)  // transacción
    Capturar error con código 23P01                 // la BD detectó un choque
        Lanzar HorarioNoDisponibleException
    FinIntentar
    Si datos.Correo <> "" Entonces
        enviado <- EnviarConfirmacion(res, datos.Correo)
        res.RegistrarEnvioCorreo(enviado)
        ActualizarCorreoEnviado(res)
    FinSi
FinFuncion
```

5.4 A3 — Iniciar sesión (pantalla) y VerificarIntento (GestorUsuarios)

```
Algoritmo IniciarSesion
    accesoConcedido <- Falso
    Repetir
        Leer U, C
        Si U = "" O C = "" Entonces
            Escribir "Completa usuario y contraseña"
        SiNo
            usuario <- BuscarPorNombreUsuario(U)
            Si VerificarIntento(usuario, C, AhoraBolivia()) Entonces
                accesoConcedido <- Verdadero
            SiNo
                Escribir MENSAJE      // "Usuario o contraseña incorrectos. Tras 3 intentos…"
            FinSi
            Si usuario <> Nulo Entonces
                GuardarIntentos(usuario)          // el contador vive en la tabla usuarios
            FinSi
        FinSi
    Hasta Que accesoConcedido
    Si usuario.Rol = Administrador Entonces
        Escribir "Panel con menú completo"
    SiNo
        Escribir "Panel de recepción"
    FinSi
FinAlgoritmo

Funcion acceso <- VerificarIntento(usuario, contrasena, ahora)
    Si usuario = Nulo O NO usuario.Activo O usuario.EstaBloqueado(ahora) Entonces
        acceso <- Falso
    SiNo
        Si VerificarHash(contrasena, usuario.ContrasenaHash) Entonces
            usuario.ReiniciarIntentos()
            acceso <- Verdadero
        SiNo
            usuario.RegistrarIntentoFallido(ahora)
            acceso <- Falso
        FinSi
    FinSi
FinFuncion

Funcion bloqueada <- RegistrarIntentoFallido(ahora)        // método de Usuario
    intentosFallidos <- intentosFallidos + 1
    Si intentosFallidos >= MaximoIntentos Entonces
        bloqueadoHasta <- ahora + MinutosBloqueo minutos
        intentosFallidos <- 0
        bloqueada <- Verdadero
    SiNo
        bloqueada <- Falso
    FinSi
FinFuncion
```

5.5 A4 — CambiarEstado y Reprogramar (GestorReservas y Reserva)

```
Funcion CambiarEstado(idReserva, nuevoEstado)
    res <- ObtenerReserva(idReserva)
    ahora <- AhoraBolivia()
    Segun nuevoEstado Hacer
        Cancelada: res.Cancelar(ahora)
        Atendida: res.MarcarAtendida(ahora)
        NoAsistio: res.MarcarNoAsistio(ahora)
        De Otro Modo: Lanzar "Acción no válida"
    FinSegun
    GuardarCambios(res)
FinFuncion

Funcion Cancelar(ahora)                                  // método de Reserva
    ExigirConfirmada()
    Si Inicio <= ahora Entonces
        Lanzar "La cita ya empezó: márcala como Atendida o No asistió"
    FinSi
    estado <- Cancelada
FinFuncion

Funcion CerrarCita(nuevoEstado, ahora)        // lo usan MarcarAtendida y MarcarNoAsistio
    ExigirConfirmada()
    Si Inicio > ahora Entonces
        Lanzar "La cita aún no empezó"
    FinSi
    estado <- nuevoEstado
FinFuncion

Funcion ExigirConfirmada()                     // método privado de Reserva (RN-13)
    Si estado <> Confirmada Entonces
        Lanzar "Solo se pueden modificar reservas confirmadas"
    FinSi
FinFuncion

Funcion Reprogramar(nuevaFecha, nuevaHora, ahora)        // método de Reserva
    ExigirConfirmada()
    Si Inicio <= ahora Entonces
        Lanzar "La cita ya empezó"
    FinSi
    fecha <- nuevaFecha
    horaInicio <- nuevaHora
    horaFin <- nuevaHora + servicio.DuracionMin minutos
FinFuncion

Funcion Reprogramar(idReserva, nuevaFecha, nuevaHora)    // método de GestorReservas
    res <- ObtenerReserva(idReserva)
    disponibles <- CalcularHorariosDisponibles(nuevaFecha, res.Servicio, res.Peluquero.Id, res.Id, Manual)
    Si NO Contiene(disponibles, nuevaHora) Entonces
        Lanzar HorarioNoDisponibleException
    FinSi
    res.Reprogramar(nuevaFecha, nuevaHora, AhoraBolivia())
    Intentar
        GuardarCambios(res)
    Capturar error con código 23P01
        Lanzar HorarioNoDisponibleException
    FinIntentar
FinFuncion
```

5.6 A5 — Generar reporte (GeneradorReportes)

```
Funcion reporte <- Generar(reservasDelRango, fi, ff, hoy)
    Si fi > ff Entonces
        Lanzar "Rango de fechas no válido"
    FinSi
    reporte <- NuevoReporte(fi, ff)                    // contadores en 0
    contServicio <- DiccionarioVacio()
    Para Cada r En reservasDelRango Hacer
        Si r.Fecha >= fi Y r.Fecha <= ff Entonces
            reporte.Total <- reporte.Total + 1
            reporte.CitasPorEstado[r.Estado] <- reporte.CitasPorEstado[r.Estado] + 1
            reporte.CitasPorDia[r.Fecha] <- reporte.CitasPorDia[r.Fecha] + 1
            Si r.Estado = Atendida Entonces
                reporte.IngresoReal <- reporte.IngresoReal + r.Precio
            FinSi
            Si r.Estado = Confirmada Entonces
                Si r.Fecha >= hoy Entonces
                    reporte.IngresoEstimado <- reporte.IngresoEstimado + r.Precio
                SiNo
                    reporte.SinMarcar <- reporte.SinMarcar + 1
                FinSi
            FinSi
            Si r.Estado <> Cancelada Entonces
                contServicio[r.Servicio.Nombre] <- contServicio[r.Servicio.Nombre] + 1
            FinSi
        FinSi
    FinPara
    lista <- ConvertirEnLista(contServicio)
    OrdenarBurbuja(lista)
    reporte.ServiciosOrdenados <- lista
FinFuncion
```

5.7 A5.1 — OrdenarBurbuja (GeneradorReportes)

```
Funcion OrdenarBurbuja(lista)
    Definir k, i, j Como Entero
    Definir aux Como ConteoServicio
    k <- Longitud(lista)
    Para i <- 0 Hasta k - 2 Con Paso 1 Hacer
        Para j <- 0 Hasta k - 2 - i Con Paso 1 Hacer
            Si lista[j].Cantidad < lista[j + 1].Cantidad Entonces
                aux <- lista[j]
                lista[j] <- lista[j + 1]
                lista[j + 1] <- aux
            FinSi
        FinPara
    FinPara
FinFuncion
```

5.8 Búsqueda lineal de clientes y validación del teléfono

```
Funcion encontrados <- BuscarEnLista(clientes, texto)       // GestorClientes, O(n)
    encontrados <- ListaVacia()
    buscado <- Minusculas(Recortar(texto))
    telefonoBuscado <- NormalizarTelefono(texto)
    Para Cada c En clientes Hacer
        coincideNombre <- Longitud(buscado) > 0 Y Contiene(Minusculas(c.NombreCompleto), buscado)
        coincideTelefono <- Longitud(telefonoBuscado) > 0 Y Contiene(c.Telefono, telefonoBuscado)
        Si coincideNombre O coincideTelefono Entonces
            Agregar(encontrados, c)
        FinSi
    FinPara
FinFuncion

Funcion t <- NormalizarTelefono(telefono)                    // Validador
    t <- QuitarEspaciosYGuiones(telefono)
    Si EmpiezaCon(t, "+591") Entonces
        t <- Subcadena(t, 4)
    FinSi
FinFuncion

Funcion valido <- EsTelefonoValido(telefono)                 // Validador
    t <- NormalizarTelefono(telefono)
    valido <- Longitud(t) = 8 Y (t[0] = "6" O t[0] = "7") Y SoloDigitos(t)
FinFuncion
```

6. Estructura del proyecto C#

```
proyecto-final/codigo/
├── BarberiaAsBajoLaManga.slnx          solución (Visual Studio Insiders / .NET 10)
├── logica/                             biblioteca de clases (Logica.csproj)
│   ├── Modelos/                        Persona, Cliente, Usuario, Servicio, Reserva, Turno,
│   │                                   HorarioAtencion, DatosReserva, ConteoServicio, Reporte
│   ├── Enumeraciones/                  EstadoReserva, Rol, OrigenReserva
│   ├── Gestores/                       GestorReservas, GestorClientes, GestorServicios,
│   │                                   GestorUsuarios, GeneradorReportes
│   ├── Utilidades/                     Reloj, Validador, Seguridad, RegistroErrores
│   ├── Datos/                          ConexionBD (Npgsql → Supabase)
│   ├── Integraciones/                  EnviadorCorreo (API de Brevo)
│   └── Excepciones/                    HorarioNoDisponibleException
├── presentaciones/WebBarberia/         ASP.NET Core MVC
│   ├── Controllers/                    Home, Reservar, Cuenta, Agenda, Reservas, Clientes,
│   │                                   Servicios, Usuarios, Reportes, Errores
│   ├── Views/                          vistas Razor, _Layout y _LayoutPanel (menú)
│   ├── wwwroot/css/site.css            estilos
│   ├── Program.cs                      configuración: MVC, sesión y clases de la lógica
│   └── appsettings.json                sin claves (van en user-secrets o en el hosting)
├── pruebas/Logica.Pruebas/             23 pruebas xUnit
└── basedatos/                          01_esquema.sql, 02_datos_iniciales.sql (Supabase)
```

Elemento	Qué contiene y por qué
`HomeController`	Redirige la página de inicio a la reserva (paso 1) y muestra la página de error general.
`Program.cs`	Registra MVC con token antiforgery en todos los formularios (RNF-04), el inicio de sesión con cookie protegida de 8 horas (RNF-03) y crea las clases de la lógica leyendo las claves de la configuración, nunca del código (RNF-05).
Menú principal	`_LayoutPanel.cshtml`: menú lateral con Agenda, Reservas, Clientes y Cerrar sesión; Servicios, Usuarios, Reportes y Registro de errores solo para el Administrador (`[Authorize(Roles = …)]`).

Elemento	Qué contiene y por qué
Base de datos	`01_esquema.sql` crea las 5 tablas (con `peluqueros` desde el 22/10; una base anterior se actualiza con `04_peluqueros.sql`) con sus validaciones (CHECK), la restricción anti-choques (EXCLUDE, código 23P01) y la seguridad por filas. Se probó en PostgreSQL: acepta citas contiguas, el mismo horario otro día y una cancelada superpuesta, y rechaza la superpuesta del mismo peluquero (otro peluquero a la misma hora sí se acepta).
Archivos de texto	`RegistroErrores` escribe y lee `App_Data/errores.txt`; la agenda y los reportes se exportan a .txt en memoria y se descargan.

Estado actual: la solución **compila sin errores ni advertencias**, la página pública muestra el paso 1 de la reserva con los servicios referenciales, el inicio de sesión responde y el panel exige iniciar sesión según el rol.

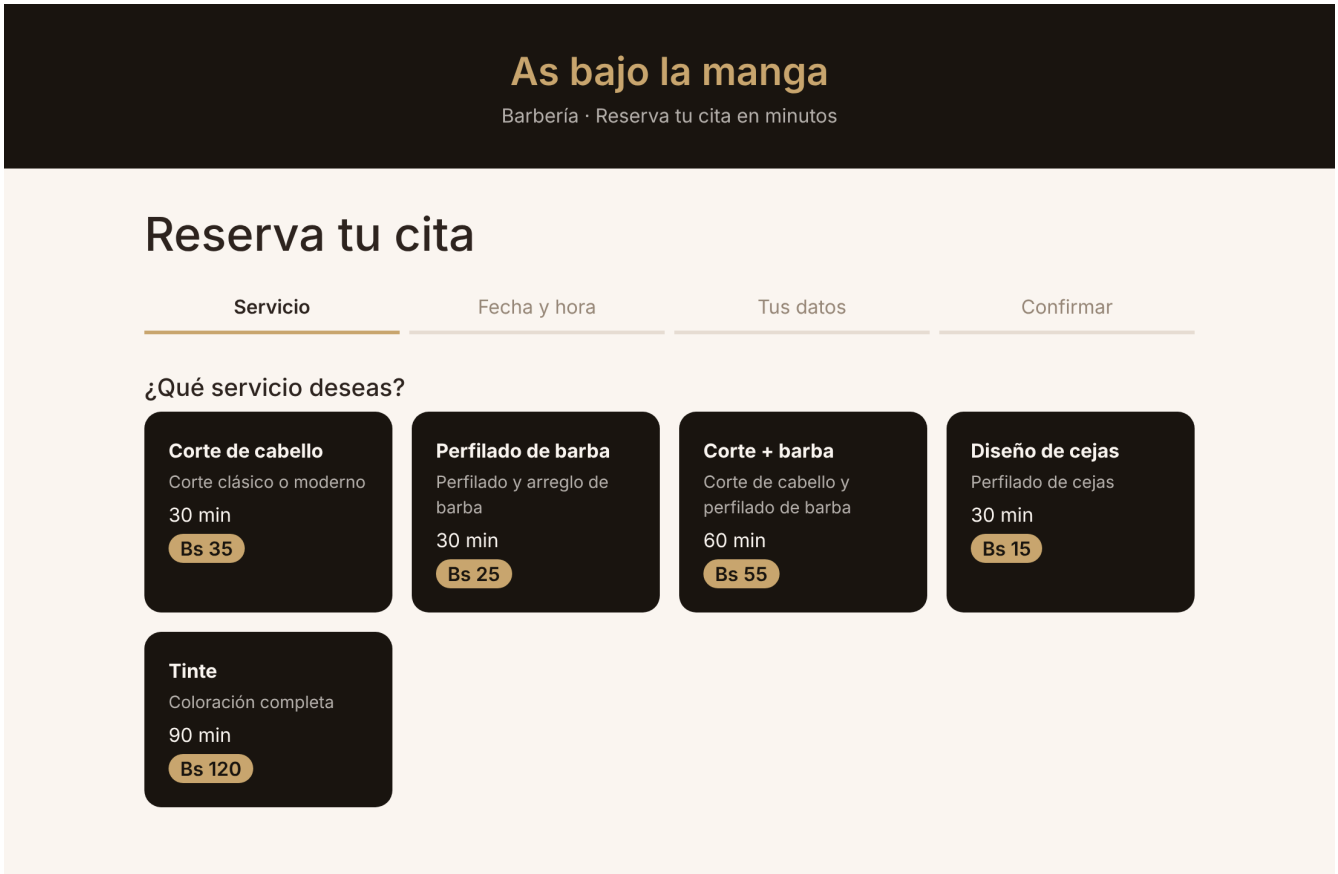


Figura 8. Paso 1 de la reserva funcionando (estructura inicial)

7. Pruebas automáticas de la lógica

Las 23 pruebas (proyecto `Logica.Pruebas`, comando `dotnet test`) **pasan**. Comprueban los algoritmos del diseño antes de conectar la base de datos:

Prueba	Qué verifica
PruebaDeEscritorio_DevuelveLosNueveHorarios	A1 con los datos de la prueba de escritorio del análisis: 09:00, 11:00, 14:00, 16:30, 17:00, 17:30, 18:00, 18:30 y 19:00.
HayChoque_CitasContiguasNoChocan	A1.1 y RN-06: las citas contiguas no chocan y las superpuestas sí.
ServicioQueCruzaElDescanso_NoSeOfrece	RN-05: un tinte de 90 min no se ofrece a las 11:00 ni a las 19:00.
ReservaCanceladaNoOcupaHorario	RN-07.

Prueba	Qué verifica
AlReprogramar_LaPropiaReservaNoSeCuenta	idExcluir en A1 (A4).
FechasNoValidas_NoTienenHorarios (3 casos) y FechaATreintaDias_SiTieneHorarios	RN-08: domingo, fecha pasada, 31 días (no) y 30 días (sí).
Hoy_LaWebExigeTreintaMinutos…	RN-09: a las 10:10 la web ofrece desde 11:00 y la reserva manual desde 10:00.
Burbuja_OrdenaDeMayorAMenor y Reporte_CuentaEstadosIngresosYServicios	A5.1 y A5.
Reserva_NoSeCancelaSiYaEmpezo… y ReservaNueva_QuedaConfirmada…	RN-13, RN-14, RN-15.
Telefono_SeValidaYNormaliza (5 casos) y Correo_Obligatorio…	Validaciones de entrada y RN-12.
Contrasena_SeVerificaConSuSal…	RNF-02: PBKDF2 con sal; nunca texto plano.
InicioDeSesion_TresIntentosFallidosBloqueanCincoMinutos	A3 y RN-18.
BusquedaLineal_EncuentraPorNombreOTelefono	Búsqueda lineal O(n).

8. Correspondencia entre algoritmo, pseudocódigo, código y prueba

Algoritmo (Etapa 2)	Método en C#	Clase	Prueba
A1 Calcular horarios disponibles	`CalcularHorariosDisponibles`	GestorReservas	PruebaDeEscritorio…, ServicioQueCruza…, FechasNoValidas…, Hoy_…
A1.1 HayChoque	`HayChoque`	GestorReservas	HayChoque_CitasContiguasNoChocan
A2 Registrar reserva	`RegistrarReserva`	GestorReservas	(V1: necesita la base de datos)
A3 Iniciar sesión	`IniciarSesion`, `VerificarIntento`, `RegistrarIntentoFallido`	GestorUsuarios, Usuario	InicioDeSesion_TresIntentos…
A4 Cambiar estado / reprogramar	`CambiarEstado`, `Reprogramar`, `Cancelar`, `MarcarAtendida`, `MarcarNoAsistio`	GestorReservas, Reserva	Reserva_NoSeCancela…, AlReprogramar…
A5 Generar reporte	`Generar`	GeneradorReportes	Reporte_CuentaEstados…
A5.1 Burbuja	`OrdenarBurbuja`	GeneradorReportes	Burbuja_OrdenaDeMayorAMenor

9. Próximos pasos (implementación, 13–16/10)

- Completar las consultas a Supabase en los gestores (métodos marcados "se implementa en la V1"), con consultas con parámetros y la transacción de A2.
- Pasos 2 a 4 de la reserva web y la pantalla «¡Cita confirmada!».
- Panel: agenda del día con exportación .txt, reserva manual, cancelar y marcar atendida / no asistió; CRUD de clientes; registro de errores.
- Antes de empezar: crear el proyecto de Supabase (región Europa), ejecutar los scripts SQL, configurar Brevo y hacer la prueba de humo en MonsterASP.NET.

Anexo. Checklist de la Presentación 2

Ítem del cronograma	Dónde está
Requerimientos	Análisis, secciones 4 y 5 (resumen en la sección 1 de este documento).
Entradas, procesos y salidas	Análisis, sección 6.
Algoritmo	Análisis, sección 7 y Anexo A (A1 a A5.1).
Pseudocódigo	Sección 5 de este documento.
Diagrama de flujo	Análisis, figuras 1 a 16.
Diagrama UML de clases (relaciones y cardinalidad)	Sección 4, figuras 2 a 7, y tabla de relaciones.
Identificación de clases, atributos y métodos	Sección 3.
Estructura inicial del proyecto C#	Sección 6 y el código en `codigo/` (compila; 23 pruebas pasan).